

Chapitre 4 - C#, SQL Server, DAO Design Pattern

I.	Problématique.....	1
II.	Solution théorique	1
III.	Solution pratique	2
1.	Méthode statique ou pas ?.....	2
2.	Classe technique AccesSGBD.....	3
3.	Classe technique SalarieDAO.....	3
4.	Modifications sur l'application	5
5.	Classes techniques AbsenceDAO et MotifDAO	7
6.	Modifications sur l'application - Suite.....	9
IV.	Conclusion	10
V.	PPE	11

I. Problématique

Au chapitre 2, on a réalisé une application (Gestion des absences des salariés) sur le modèle :



Cela signifie que les classes métier de l'application (*Salarie*, *Absence*, *Motif*) ont accès directement au stockage.

Les reproches à ce modèle sont multiples :

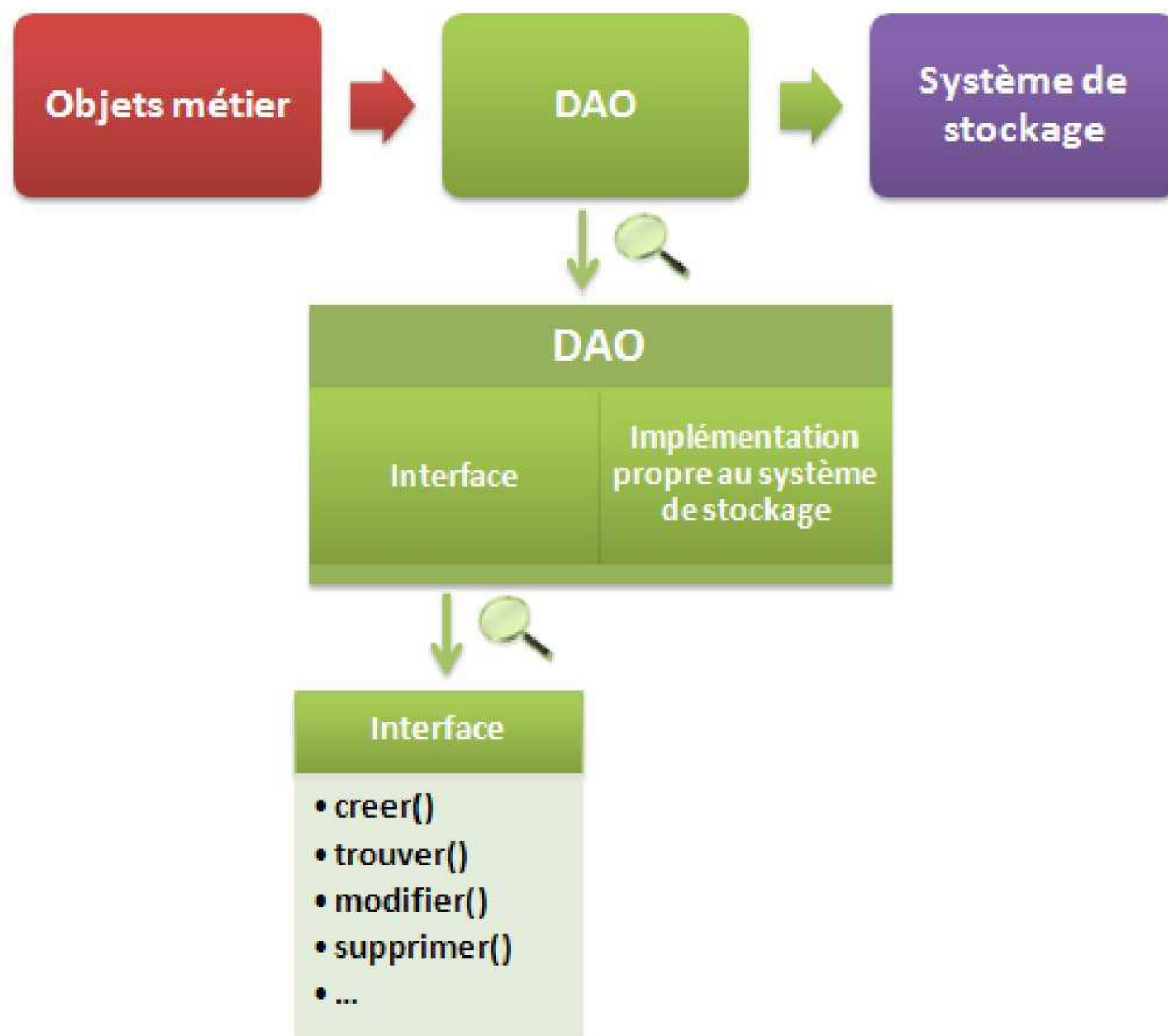
- Dépendance vis-à-vis du choix de stockage des données (base de données, fichier XML, ...)
- Redondance du code pour le stockage, notamment l'identification vis-à-vis du SGBD ;
- Inadapté pour réaliser des applications importantes (avec beaucoup de tables) ;
- Difficile à maintenir ;

II. Solution théorique

L'idée est d'ajouter des classes DAO, une pour chaque classe métier dont on veut gérer la **persistance** (capacité à garder les données dans le temps). Nouvelle organisation :



Dans chaque classe DAO, on trouvera des méthodes pour accéder aux données, les modifier, les supprimer, obtenir une liste répondant à un critère, ...



III. Solution pratique

Reprendre l'application *EpokaAbsence* du chapitre 2.

1. Méthode statique ou pas ?

Rappel : Une méthode statique est une méthode associée à la classe, sans nécessiter un objet pour l'appeler. De même, une propriété statique est liée à la classe. Exemple :

La classe *SalarieDAO* devra enregistrer un salarié au travers de la méthode :

```
public void modifierSalarie(Salarie unSalarie)
```

Si la méthode est définie non statique (par défaut), il faudra créer un objet de la classe *SalarieDAO*, juste pour pouvoir appeler la méthode *modifierSalarie* :

```
Salarie unSalarie = new Salarie();
SalarieDAO salarieDAO = new SalarieDAO();
unSalarie.no = 1;
unSalarie.nom = "DUPONT";
unSalarie.prenom = "nouveau prénom";
salarieDAO.modifierSalarie(unSalarie);
```

En revanche, avec une méthode statique :

```
public static void modifierSalarie(Salarie unSalarie)
```

Plus besoin d'objet de la classe *SalarieDAO* :


```

Salarie unSalarie = new Salarie();
unSalarie.no = 1;
unSalarie.nom = "DUPONT";
unSalarie.prenom = "nouveau prénom";
SalarieDAO.modifierSalarie(unSalarie);

```

L'appel est plus rapide. Donc pour une utilisation dite « One shot », préférer les méthodes et propriétés statiques (C'est le cas de *Int32.parse()* par exemple)

2. Classe technique AccesSGBD

Les accès au SGBD seront faits dans chaque classe DAO. Il faut éviter la redondance de l'identification :

- Ajouter une classe *AccesSGBD* : *Add, New Item, Visual C# Items, Class*. Nommer la classe *AccesSGBD* ;
- Définir une propriété publique et le constructeur de la classe :

```

class AccesSGBD
{
    public SqlConnection connexion = new SqlConnection();

    public AccesSGBD()
    {
        connexion.ConnectionString = "Data Source=(localdb)\\ProjectsV12; "
            + "Initial Catalog=Epoka_Absences; Integrated Security=SSPI;";
    }
}

```

3. Classe technique SalarieDAO

Cette classe fait le lien entre la classe métier *Salarie* et la base de données :

- Ajouter une classe DAO pour la classe métier *Salarie* nommée *SalarieDAO* ;
- Ajouter si nécessaire les clauses using suivantes :

```

using System.Data;
using System.Data.SqlClient;

```

Cette classe propose 4 méthodes, pour lire tous les salariés, et ajouter, modifier ou supprimer un salarié. **Il est important que ces méthodes aient une signature ne comportant que des classes métiers** (pas de champ de table par exemple), pour isoler la couche métier de celle de la base.

La sécurisation du code vue précédemment est utilisée.

a. Méthode lireLesSalaries

- Créer la liste de salariés renvoyée par cette méthode ;
- Utiliser la classe technique d'accès au SGBD pour se connecter ;
- Lire tous les salariés ;
- Pour chacun, créer un objet *Salarie*, le remplir et l'ajouter à la liste ;
- Refermer le curseur ;
- En cas d'erreur de type SQL, afficher un message technique ;
- Refermer systématiquement la connexion au SGBD ;
- Renvoyer la liste de salariés ;


```

public List<Salarie> LirelesSalaries()
{
    List<Salarie> listeSalaries = new List<Salarie>();
    AccesSGBD accesSGBD = new AccesSGBD();
    accesSGBD.connexion.Open();
    try {
        SqlCommand sqlCmd = new SqlCommand("SELECT * FROM Salarie", accesSGBD.connexion);
        SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
        try
        {
            while (sqlDataReader.Read())
            {
                Salarie unSalarie = new Salarie();
                unSalarie.no = (int)sqlDataReader["SalNo"];
                unSalarie.nom = (String)sqlDataReader["SalNom"];
                unSalarie.prenom = (String)sqlDataReader["SalPrenom"];
                listeSalaries.Add(unSalarie);
            }
        }
        finally
        {
            sqlDataReader.Close();
        }
    }
    catch (SqlException sqlEx) {
        Console.WriteLine ("Erreur : " + sqlEx.Message);
    }
    finally {
        accesSGBD.connexion.Close();
    }
    return listeSalaries;
}

```

b. Méthode ajouterSalarie

- Utiliser la classe technique d'accès au SGBD pour se connecter ;
- Préparer la requête d'insertion et l'exécuter ;
- Gérer les erreurs SQL et libérer l'accès au serveur ;

```

public void ajouterSalarie (Salarie unSalarie) {
    AccesSGBD accesSGBD = new AccesSGBD();
    accesSGBD.connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("INSERT INTO Salarie (SalNom,SalPrenom) VALUES "
            + "(" + unSalarie.nom + ", '" + unSalarie.prenom + "'", accesSGBD.connexion);
        sqlCmd.ExecuteNonQuery();
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        accesSGBD.connexion.Close();
    }
}

```

c. Méthode modifierSalarie

- Méthode analogue à la précédente ;


```

public void modifierSalarie(Salarie unSalarie)
{
    AccesSGBD accesSGBD = new AccesSGBD();
    accesSGBD.connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("UPDATE Salarie SET SalNom='" + unSalarie.nom
            + "', SalPrenom='" + unSalarie.prenom + "' WHERE SalNo=" + unSalarie.no.ToString(),
            accesSGBD.connexion);
        sqlCmd.ExecuteNonQuery();
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        accesSGBD.connexion.Close();
    }
}

```

d. Méthode supprimerSalarie

- Méthode analogue à la précédente ;

```

public void supprimerSalarie(Salarie unSalarie)
{
    AccesSGBD accesSGBD = new AccesSGBD();
    accesSGBD.connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("DELETE FROM Salarie WHERE SalNo="
            + unSalarie.no.ToString(), accesSGBD.connexion);
        sqlCmd.ExecuteNonQuery();
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        accesSGBD.connexion.Close();
    }
}

```

4. Modifications sur l'application

La méthode *majListeSalaries* est impactée. Elle ne comporte plus aucune référence à la base de données :

- Les deux listes sont effacées comme auparavant ;
- La classe *SalarieDAO* sert à récupérer la liste des salariés ;
- Pour chacun, les deux listes sont remplies ;


```

private void majListeSalaries()
{
    // Effacer la liste de l'onglet Salaries
    lvSalaries.Items.Clear();
    // Effectuer la dé-selection, non produite automatiquement
    lvSalaries_SelectedIndexChanged(null, null);

    // Effacer aussi la liste déroulante des salariés de l'onglet Absences
    cbSalarie.Items.Clear();
    // Effectuer la dé-selection, non produite automatiquement
    cbSalarie_SelectedIndexChanged(null, null);

    SalarieDAO salarieDAO = new SalarieDAO();
    List<Salarie> lesSalaries = salarieDAO.LirelesSalaries();
    foreach (Salarie unSalarie in lesSalaries) {
        // Remplissage de la liste de l'onglet Salaries
        ListViewItem lvi = new ListViewItem();
        lvi.Text = unSalarie.no.ToString();
        lvi.SubItems.Add(unSalarie.nom);
        lvi.SubItems.Add(unSalarie.prenom);
        lvSalaries.Items.Add(lvi);

        // Remplissage de la liste déroulante de l'onglet Absences
        ComboBoxItem cbi = new ComboBoxItem();
        cbi.code = unSalarie.no.ToString();
        cbi.libelle = unSalarie.nom + " " + unSalarie.prenom;
        cbSalarie.Items.Add(cbi);
    }
}

```

Le bouton *bAjouterSalarie* demande à :

- Utiliser la classe *SalarieDAO* ;
- Créer un objet *Salarie* initialisé avec les données saisies ;
- Appeler la méthode DAO pour effectuer l'ajout ;
- Actualiser la liste comme auparavant ;

```

private void bAjouterSalarie_Click(object sender, EventArgs e)
{
    SalarieDAO salarieDAO = new SalarieDAO();
    Salarie unSalarie = new Salarie();
    unSalarie.nom = tbNom.Text;
    unSalarie.prenom = tbPrenom.Text;
    salarieDAO.ajouterSalarie(unSalarie);
    majListeSalaries();
}

```

Les boutons *bModifierSalarie* et *bSupprimerSalarie* fonctionnent de manière similaire :

```

private void bModifierSalarie_Click(object sender, EventArgs e)
{
    SalarieDAO salarieDAO = new SalarieDAO();
    Salarie unSalarie = new Salarie();
    unSalarie.no = Int32.Parse(lvSalaries.SelectedItems[0].Text);
    unSalarie.nom = tbNom.Text;
    unSalarie.prenom = tbPrenom.Text;
    salarieDAO.modifierSalarie(unSalarie);
    majListeSalaries();
}

```



```
private void bSupprimerSalarie_Click(object sender, EventArgs e)
{
    SalarieDAO salarieDAO = new SalarieDAO();
    Salarie unSalarie = new Salarie();
    unSalarie.no = Int32.Parse(lvSalaries.SelectedItems[0].Text);
    salarieDAO.supprimerSalarie(unSalarie);
    majListeSalaries();
}
```

5. Classes techniques *AbsenceDAO* et *MotifDAO*

Rappel : La phase 1 du projet inclut la consultation uniquement des absences pour le salarié sélectionné.

L'adaptation du code pour le 2^{ème} onglet (absences) nécessite la création de la classe *AbsenceDAO* faisant le lien entre la classe métier *Absence* et la base de données :

- Ajouter une classe DAO nommée *AbsenceDAO* ;
- Ajouter si nécessaire les clauses using suivantes :

```
using System.Data;
using System.Data.SqlClient;
```

La liste des absences est présentée par motif. Il faut donc une liste des motifs, avec la classe DAO *MotifDAO* :

- Ajouter une classe DAO nommée *MotifDAO* ;
- Ajouter si nécessaire les clauses using suivantes :

```
using System.Data;
using System.Data.SqlClient;
```

a. Méthode *lireLesMotifs* (classe *MotifDAO*)

Elle est bâtie comme la méthode *lireLesSalaries* :

- Créer la liste de motifs renvoyée par cette méthode ;
- Utiliser la classe technique d'accès au SGBD pour se connecter ;
- Lire tous les motifs ;
- Pour chacun, créer un objet *Motif*, le remplir et l'ajouter à la liste ;
- Refermer le curseur ;
- En cas d'erreur de type SQL, afficher un message technique ;
- Refermer systématiquement la connexion au SGBD ;
- Renvoyer la liste de motifs ;


```

public List<Motif> LirelesMotifs()
{
    List<Motif> listeMotifs = new List<Motif>();
    AccesSGBD accesSGBD = new AccesSGBD();
    accesSGBD.connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("SELECT * FROM Motif", accesSGBD.connexion);
        SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
        try
        {
            while (sqlDataReader.Read())
            {
                Motif unMotif = new Motif();
                unMotif.no = (int)sqlDataReader["MotNo"];
                unMotif.libelle = (String)sqlDataReader["MotLibelle"];
                listeMotifs.Add(unMotif);
            }
        }
        finally
        {
            sqlDataReader.Close();
        }
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        accesSGBD.connexion.Close();
    }
    return listeMotifs;
}

```

b. Méthode lireLesAbsencesDeSalarie (classe AbsenceDAO)

Là aussi construite comme la méthode *lireLesSalaries* :

- Créer la liste d'absences renvoyée par cette méthode ;
- Utiliser la classe technique d'accès au SGBD pour se connecter ;
- Lire toutes les absences avec les motifs associés ;
- Pour chacune, créer un objet *Absence*, rempli avec *no*, *début* et *fin* ;
- Y ajouter un objet *Motif* créé pour l'occasion ;
- Y ajouter l'objet *Salarie* servant à la recherche (inutile de le dupliquer) ;
- Ajouter l'absence à la liste ;
- Refermer le curseur ;
- En cas d'erreur de type SQL, afficher un message technique ;
- Refermer systématiquement la connexion au SGBD ;
- Renvoyer la liste d'absences ;


```

public List<Absence> LirelesAbsencesDeSalarie(Salarie unSalarie)
{
    List<Absence> listeAbsences = new List<Absence>();
    AccesSGBD accesSGBD = new AccesSGBD();
    accesSGBD.connexion.Open();
    try
    {
        SqlCommand sqlCmd = new SqlCommand("SELECT * FROM Absence,Motif WHERE AbsNoMotif=MotNo "
            + "AND AbsNoSalarie=" + unSalarie.no.ToString(), accesSGBD.connexion);

        SqlDataReader sqlDataReader = sqlCmd.ExecuteReader();
        try
        {
            while (sqlDataReader.Read())
            {
                Absence uneAbsence = new Absence();
                uneAbsence.no = (int)sqlDataReader["AbsNo"];
                uneAbsence.debut = (DateTime)sqlDataReader["AbsDebut"];
                uneAbsence.fin = (DateTime)sqlDataReader["AbsFin"];
                uneAbsence.motif = new Motif();
                uneAbsence.motif.no = (int)sqlDataReader["MotNo"];
                uneAbsence.motif.libelle = (string)sqlDataReader["MotLibelle"];
                uneAbsence.salarie = unSalarie;
                listeAbsences.Add(uneAbsence);
            }
        }
        finally
        {
            sqlDataReader.Close();
        }
    }
    catch (SqlException sqlEx)
    {
        Console.WriteLine("Erreur : " + sqlEx.Message);
    }
    finally
    {
        accesSGBD.connexion.Close();
    }
    return listeAbsences;
}

```

6. Modifications sur l'application - Suite

La méthode *majListeAbsences* est aussi impactée. Elle ne comporte plus aucune référence à la base de données :

- La liste est effacée comme auparavant ;
- La classe *MotifDAO* sert à récupérer la liste des motifs ;
- Pour chacun, un groupe est créé dans la liste ;
- Si un salarié est sélectionné, la classe *AbsenceDAO* permet de retrouver les absences de ce salarié ;
- Pour chacune, une ligne est créée dans la liste, avec le rattachement au motif correspondant ;


```

private void majListeAbsences()
{
    lvAbsences.Items.Clear(); // Effacer la liste de l'onglet Absences

    // Remplir les ListViewGroup avec les motifs
    MotifDAO motifDAO = new MotifDAO();
    List<Motif> lesMotifs = motifDAO.LirelesMotifs();
    foreach (Motif unMotif in lesMotifs)
    {
        ListViewGroup lvg = new ListViewGroup(unMotif.libelle);
        lvAbsences.Groups.Add(lvg);
    }

    if (cbSalarie.SelectedItem != null)
    {
        // Remplir les ListViewItem avec les absences du salarié,
        // en les rattachant au bon ListViewGroup (motif)
        AbsenceDAO absenceDAO = new AbsenceDAO();
        Salarie unSalarie = new Salarie();
        unSalarie.no = Int32.Parse((cbSalarie.SelectedItem as ComboBoxItem).code.ToString());
        List<Absence> lesAbsence = absenceDAO.LirelesAbsencesDeSalarie(unSalarie);
        foreach (Absence uneAbsence in lesAbsence)
        {
            ListViewItem lvi = new ListViewItem();
            lvi.Text = uneAbsence.debut.ToString("ddd dd/MM/yyyy");
            lvi.SubItems.Add(uneAbsence.fin.ToString("ddd dd/MM/yyyy"));
            TimeSpan ts = uneAbsence.fin - uneAbsence.debut;
            lvi.SubItems.Add((ts.Days+1).ToString()+" jour(s)");

            // Rechercher le groupe :
            foreach (ListViewGroup lvg in lvAbsences.Groups)
            {
                if (lvg.Header.ToString() == uneAbsence.motif.libelle)
                {
                    lvi.Group = lvg;
                }
            };
            lvAbsences.Items.Add(lvi);
        }
    }
}

```

IV. Conclusion

Un code un peu plus long à écrire, mais :

- Le code est plus robuste (fiable) ;
- La maintenance / évolution est facilitée ;
- La migration vers un autre support de stockage (fichiers XML, autre SGBD, ...) est envisageable ;

Obtenu au chapitre précédent :

- Sécurité dans le code (erreur SQL interceptée) ;

Toujours pas traité :

- Absence de sécurité dans les requêtes SQL (toute requête peut être passée, l'injection SQL est possible, ...) ;

V. PPE

Vous êtes chargé de réaliser la phase 2 avec le design pattern DAO.

Prenez le projet de ce chapitre, et complétez-le avec le code déjà réalisé du chapitre précédent, que vous adapterez.

Gardez bien des projets distincts pour chaque chapitre.